

Software Design Specification

Document Information

Category	Information
Customer	UOS – Department of Computer Science
Project	<Hotel Room Management >
Document	Software Design Specification
Document Version	1.0
Identifier	<PGBH01-2025-DS>
Status	Draft
Author(s)	<Daud George, Kamran Haider, Muhammad Manan>
Approver(s)	PM
Issue Date	Sept. 15, 2025
Document Location	
Distribution	1. Advisor 2. PM 3. Project Office

Definition of Terms, Acronyms and Abbreviations

Term / Acronym	Definition
HRMS	Hotel Room Management System; software for managing hotel operations.
Room Inventory	Total rooms available in the hotel including type, status, and features.
Room Status	Current state of a room: Available, Occupied, Reserved,

	Dirty. Maintenance.
Reservation	A guest booking for specific dates including personal and stay details.
Check-In	Process where a guest arrives, is registered, and assigned a room.
Check-Out	Process of ending a guest stay and settling payments.
Guest Profile	Stored information about a guest including name, contacts, and booking history.
Folio	A document containing all charges and transactions for a guest's stay.
Invoice	Final bill provided to the guest at check-out.
Rate Plan	Pricing structure for rooms (seasonal, promotional, corporate, etc.).
OTA	Online Travel Agency (Booking.com, Expedia) used for external reservations.
API	Application Programming Interface; allows communication between software systems.
UI	User Interface; screens users interact with in the system.
UX	User Experience; overall feel and ease of using the system.
PMS	Property Management System; manages hotel operations and integrates with HRMS.

Table of Contents

<u>1.Introduction</u>	4
<u>1.1Purpose of Document</u>	4
<u>1.2Project Overview</u>	4
<u>1.3Scope</u>	4
<u>2.Design Considerations</u>	4
<u>2.1Assumptions and Dependencies</u>	4
<u>2.2Risks and Volatile Areas</u>	4
<u>3.System Architecture</u>	4
<u>3.1System Level Architecture</u>	4
<u>3.2Sub-System / Component / Module Level Architecture</u>	5
<u>3.3Sub-Component / Sub-Module Level Architecture (1...n)</u>	5
<u>4.Design Strategies</u>	5
<u>4.1Strategy 1...n</u>	5
<u>5.Detailed System Design</u>	5
<u>6.References</u>	6
<u>7.Appendices</u>	6

1. Introduction

1.1 Purpose of Document

The purpose of this document is to explain how the Hotel Room Management System (HRMS) will be designed. It describes how the system will work, what parts it will have, and how those parts will connect with each other. This document guides the development team so they can build the system correctly according to the needs of the hotel.

It also helps everyone involved in the project understand the system's structure before development begins. By reading this document, the team will have a clear idea of the system's design, features, and flow of information.

Intended Audience

This document is meant for:

- Developers: who will write the code of the system.
- Testers: who will check the system for errors and problems.
- Project Managers: who will manage the progress of the project.
- Hotel Staff or Stakeholders: who want to review and confirm that the system matches their needs.
- Support Team: who may maintain or update the system later.

Design Methodology Used

This project uses an Object-Oriented Design (OOD) approach.

This means the system is designed using real-world objects like Rooms, Guests, Reservations, and Invoices. Each object has its own properties and actions. This method makes the system:

- easier to understand
- easier to update
- easier to reuse
- more organized and flexible

1.2 Project Overview

- The Hotel Room Management System (HRMS) is a software application designed to help hotels manage their daily activities more easily and efficiently. Instead of recording bookings, room details, and guest information on paper or spreadsheets, this system provides a digital and organized way to handle all hotel operations.

- The system allows hotel staff to manage room availability, make reservations, check guests in and out, generate bills, and track housekeeping tasks. By using this system, hotel operations become faster, more accurate, and less dependent on manual work. It also reduces the chances of double bookings, missing records, and billing mistakes.
- The HRMS supports different types of users such as receptionists, hotel managers, housekeeping staff, and administrators. Each user can access the features they need to perform their job effectively. The system also stores all data in a secure database so it can be easily accessed, updated, and maintained.
- Overall, the main aim of this project is to improve the hotel's workflow, enhance guest experience, and provide a reliable and automated way to manage rooms and hotel services.

1.3 Scope

The Hotel Room Management System (HRMS) is designed to help hotels manage their rooms, guests, and daily operations in an organized and efficient way. It covers the major tasks done by hotel staff and provides a digital platform to handle them.

What the System Will Do ?

1. Manage Room Information
 - a. Store details of rooms (type, status, price, features).
 - b. Update room status (Available, Occupied, Dirty, Maintenance).
2. Handle Reservations
 - a. Allow staff to create, update, or cancel bookings.
 - b. Check room availability for selected dates.
3. Check-In / Check-Out Process
 - a. Register guests at arrival.
 - b. Assign rooms.
 - c. Complete check-out with billing.
4. Guest Management
 - a. Store guest details (name, contact, ID).
 - b. Maintain guest history.

5. Billing and Payments
 - a. Generate bills/invoices.
 - b. Record payments (cash, card, online methods where applicable).
6. Housekeeping Management
 - a. Track room cleaning status.
 - b. Create and update housekeeping tasks.
7. Reports Generation
 - a. Daily arrivals and departures list.
 - b. Room occupancy reports.
 - c. Revenue and billing reports.

What the System Will Not Do (Out-of-Scope)

1. Online Booking from OTAs

Direct integration with Booking.com, Airbnb, Expedia, etc., is not included.
2. Advanced Financial Accounting

A full accounting system (profit/loss, tax returns, payroll) is not provided.
3. Restaurant or POS System

Food ordering, restaurant billing, and kitchen management are not part of this project.
4. Mobile Application

The system does not include a mobile app for guests or staff.

2. Design Considerations

2.1 Assumptions and Dependencies

These are points that the design is based on and are expected to remain true:

- The hotel has a stable internet or local network for the system to work.
- Users (receptionists, managers, housekeeping) have basic computer knowledge.
- The hotel operates from a single location (not a hotel chain).
- All room and guest data will be stored in a central database.
- Users will access the system through desktop or laptop computers.
- Only hotel staff will use the system; guests will not directly interact with it.

Dependencies

These are things the system may rely on:

- Database System: A working database is required to store room, guest, and reservation information.
- Login/Authentication Module: Users must be verified before they can use the system.
- Hardware: Computers with browsers and a stable network connection.
- Operating System: Windows or any OS that supports web applications.
- External Billing Hardware (Optional): Printer for invoices or receipts.

2.2 Risks and Volatile Areas

These are potential problems that may occur:

- Human errors: Incorrect data entered by staff may cause wrong billing or double bookings.
- System downtime: If the system or internet goes down, hotel operations may slow.

- Data loss: Without proper backup, system failures could cause loss of important information.
- Unauthorized access: If passwords are weak, sensitive data may be misused.

Volatile Areas (Areas Likely to Change)

- Room Pricing Rules
Prices may change often due to seasons, discounts, or hotel policies.
- Booking Policies
Cancellation rules, check-in/out timings, or extra-guest policies may be updated.
- Payment Methods
New payment options (e.g., mobile wallets) may be added or old ones removed.
- User Roles and Permissions
Hotel may add new staff roles or change what each role can do.
- Report Format Requirements
Management may request new types of reports or change report formats.
- Integration Requirements
The system may later need to connect with external apps like taxation systems, online booking platforms, or POS systems.
- UI/UX Design Changes
Hotel may want new themes, layouts, or additional buttons or workflows.
- Regulatory or Legal Requirements
New government rules about guest data or payments may require system changes.

3. System Architecture

3.1 System Level Architecture

he HRMS is divided into the following main subsystems/modules:

1. User Interface (UI) Module
 - a. Provides screens for hotel staff (receptionists, managers, housekeeping).
 - b. Includes dashboards, booking forms, check-in/out screens, and reports.
2. Reservation and Booking Module
 - a. Handles creating, updating, and cancelling bookings.
 - b. Checks room availability and manages reservation records.
3. Room and Inventory Management Module
 - a. Maintains room details, types, status, and features.
 - b. Updates room status (Available, Occupied, Dirty, Maintenance).
4. Guest Management Module
 - a. Stores guest information, history, and ID verification details.
5. Billing and Payment Module
 - a. Generates invoices and handles payments (cash, card, etc.).
 - b. Maintains folio details for each guest stay.
6. Housekeeping Module
 - a. Tracks cleaning tasks and room maintenance.
 - b. Assigns tasks to staff and updates room status.

Authentication and Security Module

Manages login, user roles, and access permissions.

Ensures secure access to sensitive data.

Database Module

Stores all data including rooms, reservations, guests, billing, and reports.

Central repository for all hotel data.

Relationships Between Elements

- UI Module communicates with Reservation, Room, Billing, and Housekeeping Modules to display data and accept user input.
- Reservation Module interacts with Room Module to check availability.
- Billing Module depends on Reservation Module and Guest Module for calculating charges.
- Housekeeping Module updates Room Module with cleaning and maintenance status.
- Reports Module retrieves data from all other modules for analysis.
- Authentication Module controls access to all other modules.
- Database Module is shared by all modules to store and retrieve data.

Interfaces to External Systems

- Optional Payment Gateway for processing card or online payments.
- Optional Email/SMS Service for sending confirmations or notifications.
- Optional Integration with OTAs (Booking.com, Expedia) for external reservations.

Note: External interfaces are planned but not part of the initial version of the system.

4. Major Physical Design Issues

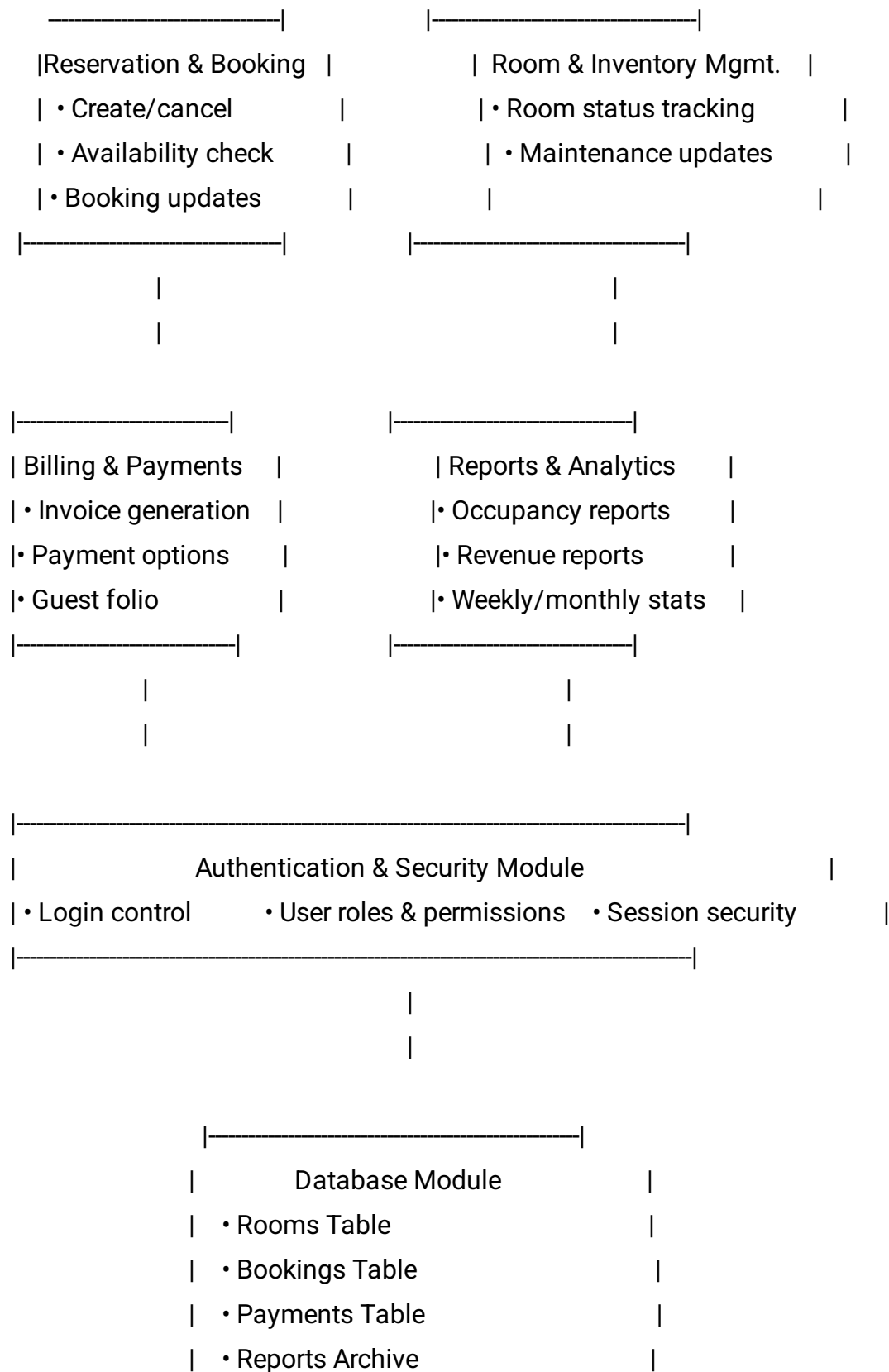
- Modules like UI, Reservation, Billing, Housekeeping, and Reports will run on a central server or web application server.
- The Database will run on a dedicated database server or cloud-managed database service.
- Users (staff) will access the system via desktop/laptop computers connected through LAN or internet.
- Optional notification services can run as separate background services or tasks.

3.2 Sub-System / Component / Module Level Architecture

User Interface (UI Module)

- Staff Dashboard
- Booking Screen
- Check-in/Check-out Forms
- Housekeeping Panel

interacts with



Subsystems Identified

1. User Interface Module

- Dashboards
- Forms
- Data entry screens

2. Reservation & Booking Subsystem

- Booking creation/cancellation
- Availability checking
- Calendar management

3. Room & Inventory Management Subsystem

- Room information
- Room status update
- Maintenance & services

4. Guest Management Subsystem

- Guest details
- Guest stay history
- Verification records

5. Billing & Payment Subsystem

- Invoice creation
- Payment processing
- Guest folio management

6. Reports & Analytics Subsystem

- Occupancy reports
- Revenue reports
- Performance statistics

7. Authentication & Security Subsystem

- Login
- Role-based access
- Data security

8. Database Subsystem

- Central data storage
- Data retrieval
- Data consistency & backup

3.3 Sub-Component / Sub-Module Level Architecture (1...n)

Sub-Modules

1. Availability Checker
2. Booking Manager
3. Booking Calendar
4. Cancellation Handler
5. Check-in/Check-out Handler

Reservation & Booking Module

Availability

Checker

Booking Manager

Calendar

Cancellation

|

Handler

Check-in/out

Handler

4. Design Strategies

This section explains the major design strategies and decisions that shape how the Hotel Room Management System (HRMS) is organized. These strategies guide how different parts of the system work together, how data is managed, and how the system grows or handles issues.

1. Modular and Layered Architecture

- The system is divided into clear modules such as Booking, Billing, Rooms, Housekeeping, Reports, etc.
- Each module performs its own function and communicates with others through defined interfaces.
- This makes the system easy to maintain, test, and update without affecting other modules.

2. Object-Oriented Design (OOD)

- Classes and objects are used to represent real-world entities (e.g., Room, Guest, Booking, Invoice).
- Principles like inheritance, encapsulation, and polymorphism help reduce complexity.
- This improves reusability, flexibility, and clarity in system structure.

3. Centralized Database Strategy

- All system modules share a single database to ensure consistency and accuracy of data.
- Room details, guest information, bookings, and payments are stored in a structured manner.
- Relationships between tables maintain data integrity and avoid duplication.

4. Separation of Concerns

- Business logic, presentation logic (UI), and database operations are separated.
- This ensures that changing the UI or database will not break core functionality.
- Makes future updates easier (e.g., adding a mobile app).

5. Standardized Interfaces for Communication

- Modules interact through clearly defined APIs or function calls.
- This reduces dependency issues and ensures smooth communication between subsystems.
- Helps in integrating external services like payment gateways or email systems.

6. Error Handling Strategy

- All modules include proper input validation and error messages.
- Errors are logged for later review by administrators.
- The system ensures that failures (e.g., payment errors, room unavailability) are handled gracefully without crashing.

7. Security and Access Control Strategy

- Role-based access is used (Admin, Receptionist, Manager, Housekeeping Staff).
- Sensitive data is encrypted and access is restricted.
- Login and user session management prevent unauthorized access.

8. Scalability Strategy

- System components are designed so that more features (like online booking or loyalty programs) can be added later.
- Database tables are normalized to support large data growth.
- Architecture can handle more rooms, more staff, and more transactions when needed.

9. Use of Reusable Components

- Common functions (user login, data validation, notifications) are designed as reusable modules.

- Reduces development time and avoids writing the same code again.

10. Performance Optimization Strategy

- Frequently used data (room availability, bookings) may be cached for faster access.
- Database queries are optimized to avoid slow performance.
- System minimizes unnecessary data loading to reduce processing time.

11. User-Friendly Interface Strategy

- UI is designed to be simple and easy to use for hotel staff.
- Clear menus, color coding for room statuses, and easy navigation improve usability.
- Reduces staff training effort and errors.

12. Logging and Monitoring Strategy

- System logs important actions such as check-ins, payments, and cancellations.
- Helps in auditing, debugging, and maintaining the system.

4.1 Strategy 1...n

Strategy 1: Modular Design

Reasoning:

Breaking the system into modules (Booking, Billing, Rooms, Users) makes it easier to develop and maintain.

Trade-offs:

More modules require careful coordination.

Covers:

- Future extension New modules can be added easily
- System reuse Common features can be reused across modules

Strategy 2: Object-Oriented Approach

Reasoning:

Using classes (Room, Guest, Booking) makes the system closer to real-world understanding and easier to expand.

Trade-offs:

Initial design takes more planning.

Covers:

- Future enhancement Adding new object types is simple
- Data management Data handled through objects

Strategy 3: Centralized Database

Reasoning:

All system data is stored in one database to keep information consistent and accurate.

Trade-offs:

If database fails, all modules can be affected.

Covers:

- Data management Easy storage and retrieval
- Concurrency Multiple users access the same data safely

Strategy 4: Simple and Clean User Interface

Reasoning:

Staff should easily understand the system without training.

Trade-offs:

Advanced features may be limited to keep it simple.

Covers:

- UI paradigms Clear menus, simple screens
- Reuse UI components reused across modules

Strategy 5: Error Handling and Logging

Reasoning:

System should not crash on mistakes. Errors must be shown clearly and recorded.

Trade-offs:

More logging needs extra storage.

Covers:

- Future enhancement Easier debugging
- Concurrency Prevents system lockups

Strategy 6: Security and Access Control

Reasoning:

Different staff roles need different access levels for safety.

Trade-offs:

More security increases setup time.

Covers:

- Data management Protects stored data

